Fundamentos de Testing, Verificación y Validación

Arquitectura de la Calidad en el Ciclo de Vida del Software

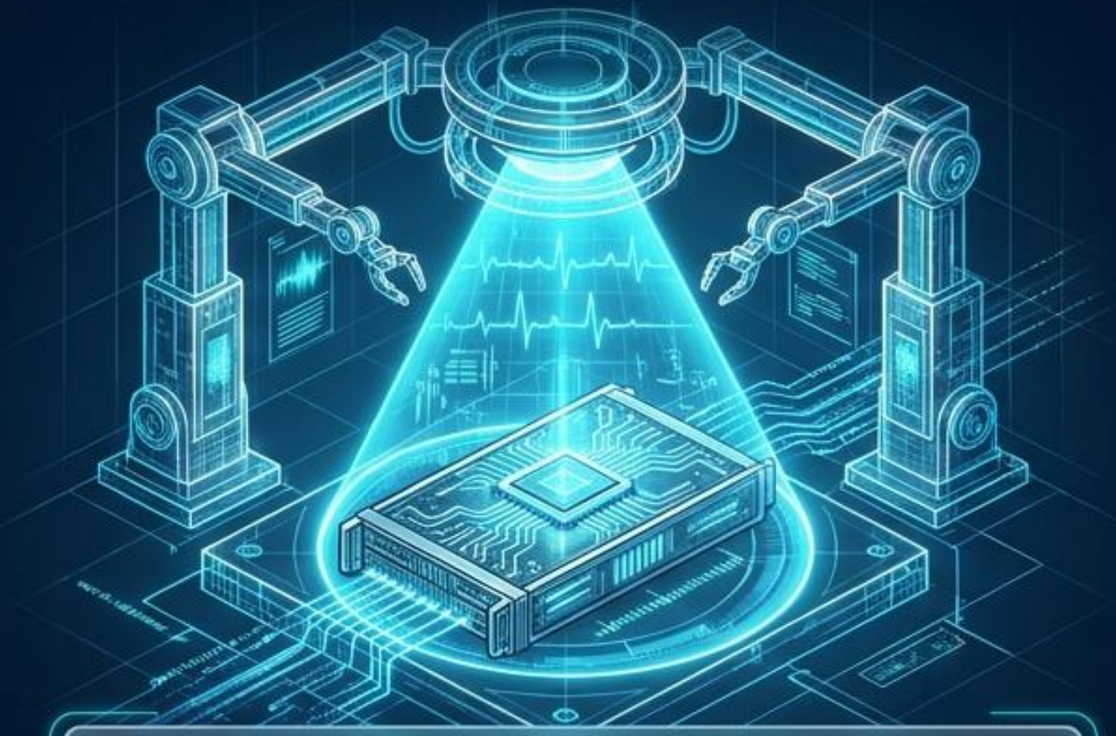
Universidad de La Sabana



Maestría en Profundización en Ingeniería de Software – Unidad 1

César Augusto Vega Fernández

La Medicina Preventiva del Software



Ninguna prueba es suficiente por sí sola.

Así como un diagnóstico médico exige radiografías, pruebas de esfuerzo y exámenes de sangre, el software requiere **múltiples niveles de evaluación**. No basta con encender el motor; hay que llevarlo a la pista.

El Problema Real



- **Errores ocultos** que destruyen el entorno de producción.
- **Sistemas colapsados** por fallos en acoplamiento y dependencias.
- **Carencia de validación**: el sistema compila perfectamente, pero nadie puede usarlo.

Verificación

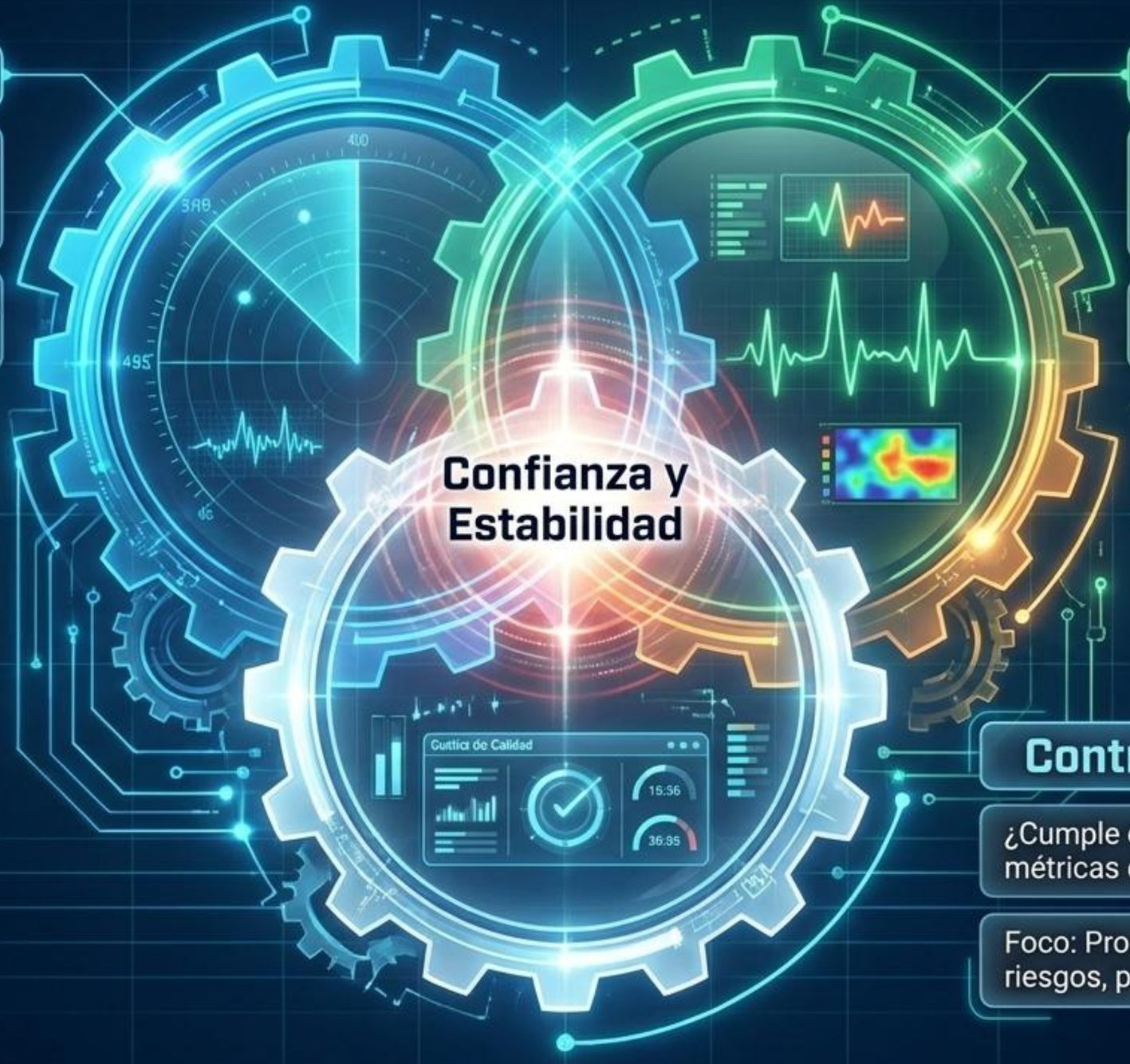
¿Estamos construyendo el software correctamente?

Foco: Reglas, diseño técnico, componentes internos.

Validación

¿Estamos construyendo el software adecuado para los usuarios?

Foco: Propósito, contexto, experiencia humana.

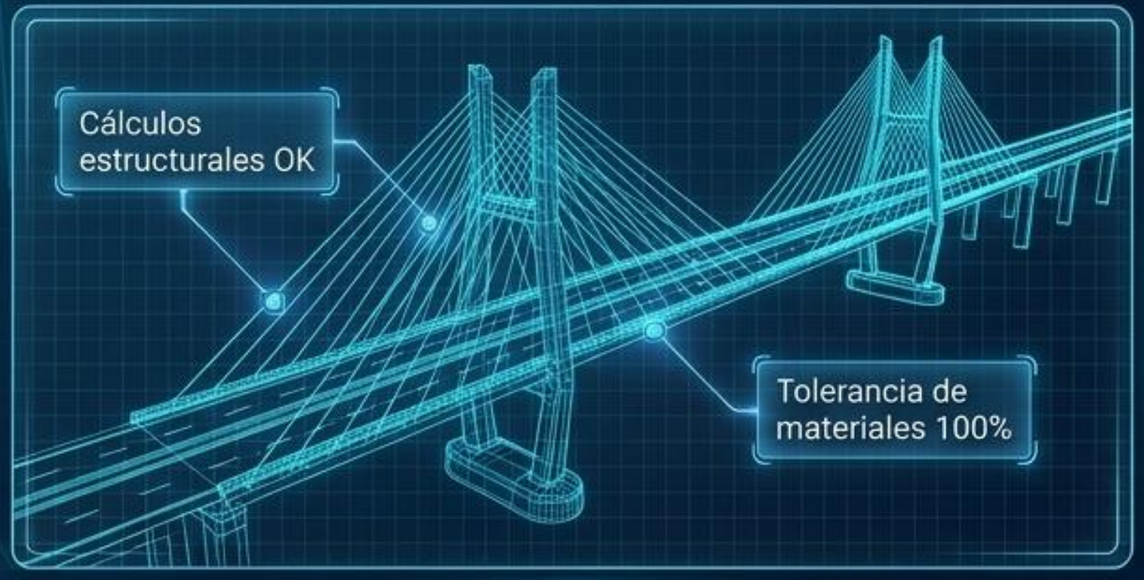


Control de Calidad

¿Cumple con los estándares y métricas definidas?

Foco: Procesos, mitigación de riesgos, prevención de defectos.

Verificación (La Forma)



(El producto está bien hecho)

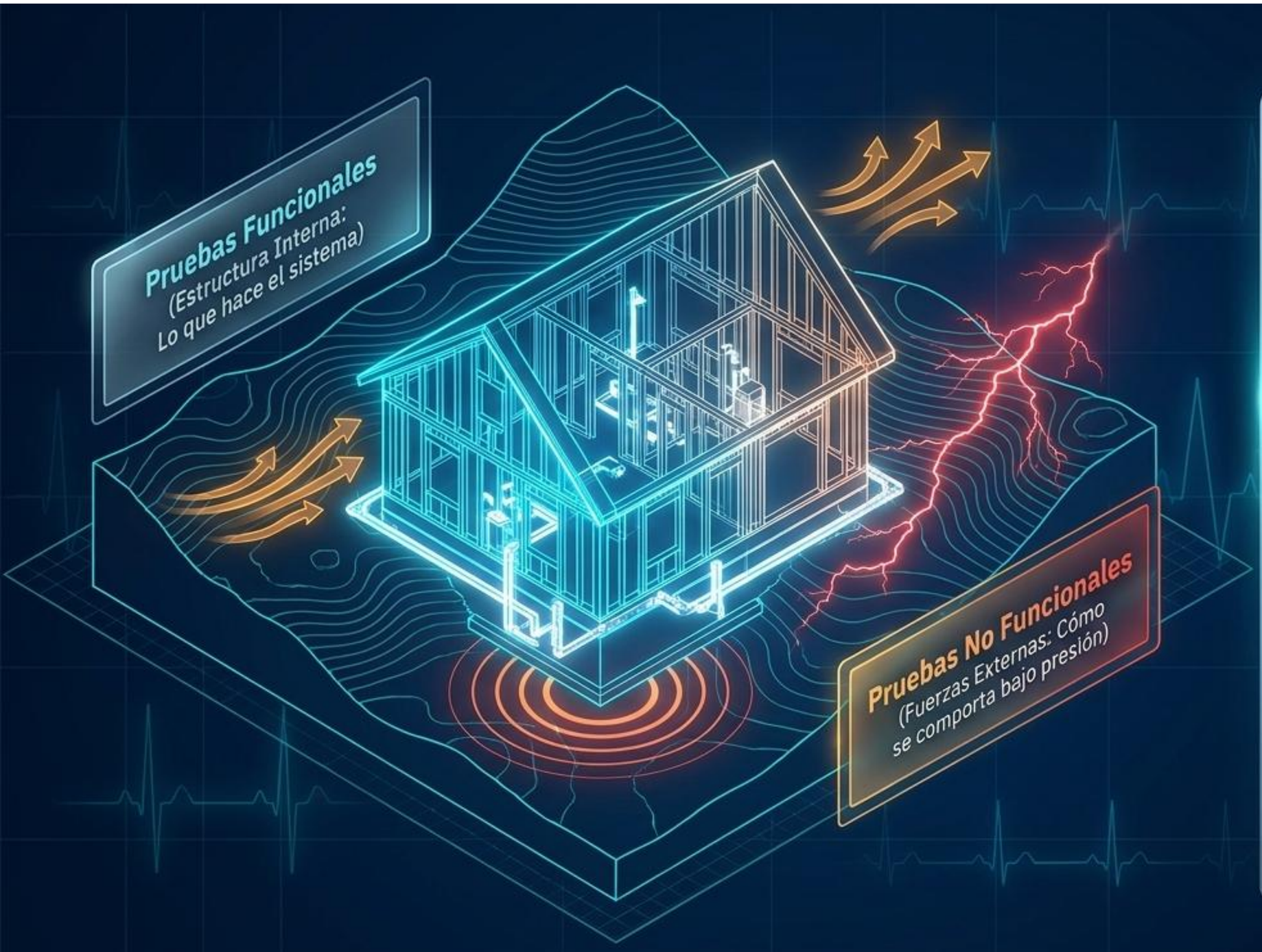
Validación (El Propósito)



(El producto no resuelve la necesidad)

Diagnostic Comparison Matrix

Dimensión	Verificación (La Forma)	Validación (El Propósito)
Naturaleza	Proceso técnico basado en reglas	Proceso enfocado en la experiencia y negocio
Ejemplo	validateAge() rechaza edad < 18	Los mensajes de error guían al usuario
Ejecución	Altamente automatizable (Scripts)	Requiere juicio y exploración humana



Axioma del Testing:

Los tipos de pruebas constituyen todo un ecosistema simbiótico.

Ninguno sobrevive de forma aislada, especialmente en arquitecturas distribuidas y microservicios.

Pruebas Unitarias y TDD

Si el ladrillo está roto, la pared también lo estará.

El Propósito

Evaluar fragmentos mínimos de código (funciones, clases, métodos) de forma completamente aislada. Es el primer muro de contención.

La Metodología TDD



Automatización Temprana

Uso intensivo de Frameworks (Ej. JUnit) y técnicas de simulación (Mocking) para evitar dependencias lentas de bases de datos.





Pruebas de Integración (El Ensamblaje)

Detectar fricciones e incompatibilidades entre contratos mal definidos.

Caso de Uso: Validar el flujo de datos precisos entre APIs y motores de bases de datos antes de construir la interfaz.



Pruebas de Sistema (El Flujo Completo)

Validación End-to-End con datos, configuración y usuarios reales operando en conjunto.

Caso de Uso: El usuario entra, se validan credenciales, se procesa en base de datos y se retorna la respuesta final.

Pruebas de Aceptación (UAT)

A pesar de que el ingeniero jure que los planos están perfectos, quien decide si la casa es habitable es el dueño.

Responsabilidad Absoluta

Ejecutadas exclusivamente por el cliente, usuario final o dueños del área de negocio.

Criterios de Aceptación

Actúan como el checklist definitivo. Si no se cumplen, el producto no sale a producción, sin importar el éxito de la verificación técnica.

Evidencia Requerida

La validación exige juicios sobre la estética, la utilidad real y el valor de negocio entregado, yendo mucho más allá de respuestas de servidor 200 OK.



Pruebas No Funcionales: El Sistema Bajo Presión

Rendimiento y Carga



Métrica: Solicitudes por segundo (RPS) y tiempo de latencia.
Objetivo: Comportamiento bajo concurrencia masiva simulada (JMeter / K6).

Estrés (Punto de Quiebre)



Métrica: Puntos de colapso absoluto.
Objetivo: Llevar el sistema hasta la falla inevitable para diseñar mecanismos de resiliencia y recuperación.

Estabilidad (Endurance)

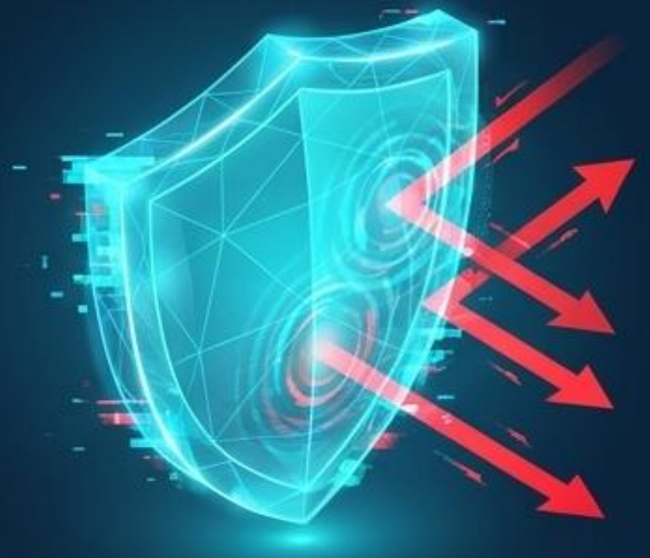


Métrica: Fugas de memoria y degradación temporal.

Objetivo: Someter el sistema a carga moderada continua durante días para detectar desgaste oculto.

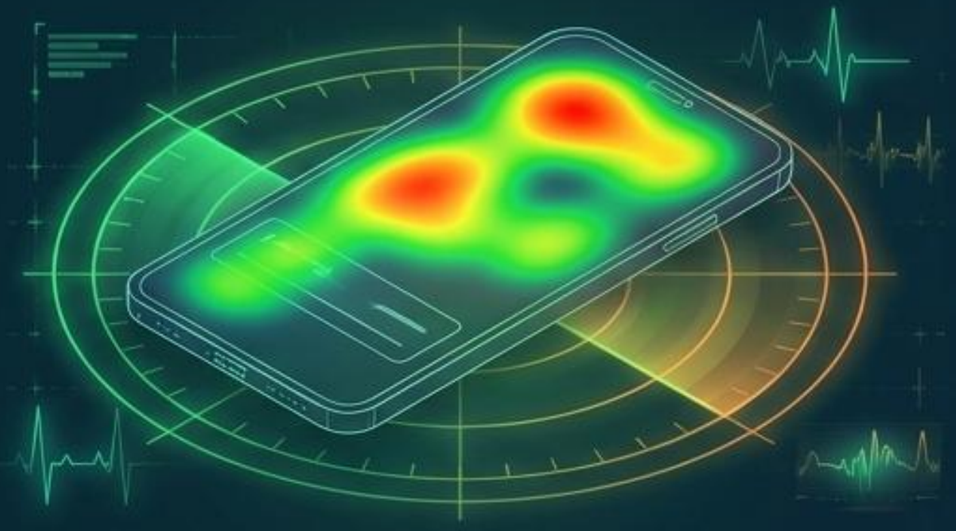
Pruebas de Seguridad y Usabilidad

Seguridad (La Fortaleza)



- Alineación con vulnerabilidades OWASP.
- Auditoría de Inyección SQL, XSS, CSRF y exposición de datos.
- Garantizar autenticación y blindaje de la capa de servicios.

Usabilidad (La Fricción Humana)



- Evaluación de accesibilidad y coherencia visual.
- Claridad contextual en mensajes de error.
- Concepto Clave: Una API puede responder en 5ms, pero si el usuario no entiende la interfaz, el sistema fracasa comercialmente.

La automatización es el sistema inmunológico del software, pero no reemplaza el juicio humano.



LA REGLA DE ORO: Automatiza lo repetitivo y objetivo. Evalúa manualmente lo complejo y subjetivo.

- ☑ Regresiones masivas
- ☑ Pruebas de carga (JMeter)
- ☑ APIs y Unitarias (JUnit)

- ☑ Exploración de UX
- ☑ Pruebas de usabilidad
- ☑ Percepción de diseño

Caso Real: El Colapso Productivo Estatal

Autopsia de un fracaso por mala priorización.



Hallazgo Temprano

Se detecta un defecto de rendimiento mediante pruebas de carga controladas en etapa de pre-producción.

Mala Priorización

El bug es clasificado erróneamente como "prioridad baja". El equipo asume que el volumen teórico no ocurrirá en la práctica.

El Lanzamiento

El sistema estatal de pagos se despliega. El tráfico de usuarios reales supera brutalmente todas las previsiones operativas.

Colapso Total

La represa se rompe. El sistema se paraliza por completo el mismo día del lanzamiento, generando daño económico y reputacional.



Lección: **Ignorar grietas pequeñas en la infraestructura preventiva provoca colapsos incontrolables bajo presión real.**

Gestión de Defectos: Mantenimiento de la Vía

Reparar las grietas de la ciudad antes de que destruyan los vehículos.

1. Registro → 2. Asignación → 3. En Progreso → 4. En Validación → 5. Cierre/Reapertura



Líder Técnico (El Orquestador) - Prioriza separando severidad de urgencia. Coordina despliegues y certifica que la vía es transitable.

QA (El Inspector) - Detecta el hueco. Documenta evidencia técnica replicable (Ej. Endpoint retorna 500 con edad=-1).

Desarrollador (El Ingeniero) - Analiza la causa raíz profunda e implementa la reparación estructural en el código.

Integración y Entrega Continua (CI/CD)



Aha Moment: **Las pruebas no son una fase final secuencial; son el fluido continuo que rodea, impregna y protege todo el desarrollo (DevSecOps).**

Conclusión: De Cazadores de Bugs a Arquitectos de Calidad

Planificación Estratégica

Probar sin planificar es construir un puente sin cálculos. Las métricas objetivas y el diseño técnico deben reemplazar la intuición.

El Nuevo Rol Profesional

El Tester moderno ya no se limita a romper código al final. Ahora evalúa riesgos integrales, diseña frameworks de automatización y define culturas DevOps.



El Valor Final

Una gestión disciplinada y metodológica transforma la incertidumbre en confianza, garantizando soluciones de software robustas, inquebrantables y radicalmente centradas en la experiencia humana.